INSTITUTO CERVANTES

Carrera: [completar]

Everyone Needs To Smile

Plataforma de peticiones vecinales con procesamiento de datos y

Medición de impacto

Historial de revisión

Versión	Fecha y hora	Autor	Detalle de la modificación
1.0	[fecha] [hh:mm]	Equipo	Propuesta original: plataforma solidaria de donaciones (Project Charter, alcance, EDT, plan de comunicación, Gantt, product backlog, arquitectura).
2.0	[fecha] [hh:mm]	Equipo	Reorientación del proyecto: se reemplaza el módulo de pagos y donaciones por un sistema de peticiones con clasificación automática, firmas, score de urgencia, detección de similares y reportes.
2.1	12/10/2026 [hh:mm]	Andrada, Juan Pablo	Actualización de todos los entregables al nuevo alcance y documentación del sistema implementado (arquitectura, modelo de datos, seguridad, pruebas).

Cada integrante que modifique el documento agrega una fila con la versión, la fecha y hora, su nombre y el cambio realizado.

Cátedra	Práctica Profesionalizante
Docente	Ing. Loza
Empresa / proyecto	ENTS — Everyone Needs To Smile
Trabajo práctico	Documentación del proyecto y entregables de gestión
Fecha de entrega	[__/__/____]

Autores

Jiménez, Ignacio	Legajo 28177
Andrada, Juan Pablo	Legajo 27318
Patrón, Máximo	Legajo 27311

Sitio publicado: entscordoba.netlify.app

Tabla de contenido

1. Introducción
2. Objetivos
3. Contenido del trabajo práctico
    3.1 Contexto, estudio de necesidades y reorientación
    3.2 Project Charter
    3.3 Declaración de alcance (Scope Statement)
    3.4 Estructura de desglose del trabajo (EDT / WBS)
    3.5 Lista de actividades
    3.6 Diccionario de actividades
    3.7 Plan de comunicación
    3.8 Diagrama de Gantt
    3.9 Requerimientos y product backlog
    3.10 Arquitectura del sistema
    3.11 Modelo de datos
    3.12 Procesamiento automático de datos
    3.13 Autenticación, roles y seguridad
    3.14 Reportes y evaluación de impacto
    3.15 Pruebas y calidad
    3.16 Ambiente de implementación y despliegue
4. Bibliografía
5. Fuentes de información
6. Anexo

1. Introducción

Everyone Needs To Smile (ENTS) es una plataforma web en la que los vecinos de la ciudad de Córdoba y de Villa Allende publican problemas de su barrio —salud, seguridad, infraestructura, servicios, vivienda, ambiente, entre otros—, los apoyan con su firma y siguen cómo los resuelven los organismos responsables.

El proyecto nació como una plataforma solidaria de donaciones. Durante el análisis, el equipo concluyó que el módulo de pagos agregaba complejidad desproporcionada (integración con pasarelas, cumplimiento legal y trazabilidad de fondos) frente al valor que aportaba. Por eso se reorientó hacia un **sistema de peticiones ciudadanas con medición de impacto**: no un formulario de reclamos, sino un sistema que procesa los datos —clasificación automática por categoría y zona, score de urgencia, detección de peticiones similares y reportes exportables— para que los organismos sepan qué atender primero y puedan registrar la respuesta, cerrando el ciclo.

Este documento presenta los entregables de gestión del proyecto actualizados al nuevo alcance (Project Charter, declaración de alcance, EDT, lista y diccionario de actividades, plan de comunicación y diagrama de Gantt) y la documentación técnica del sistema efectivamente implementado y publicado: requerimientos, arquitectura, modelo de datos, algoritmos de procesamiento, seguridad, pruebas y ambiente de implementación.

Consideramos que el trabajo es relevante porque convierte reclamos dispersos en información ordenada por barrio, urgencia y apoyo vecinal, que tanto la comunidad como las áreas del municipio pueden consultar, priorizar y responder.

2. Objetivos

2.1 Objetivo general

Desarrollar y poner en funcionamiento una plataforma web de peticiones vecinales para Córdoba y Villa Allende que permita a los vecinos visibilizar problemáticas de su barrio, apoyarlas con firmas y seguir su resolución, y que procese esa información para que los organismos públicos y las organizaciones sociales prioricen y registren su respuesta.

2.2 Objetivos específicos

1. Permitir crear peticiones con texto, fotos y ubicación, clasificándolas automáticamente por categoría y barrio.
2. Registrar firmas de apoyo y ordenar las peticiones por prioridad según la cantidad de firmas.
3. Calcular automáticamente un score de urgencia de 0 a 100 a partir de la gravedad, el riesgo, el apoyo y su velocidad.
4. Detectar peticiones similares para concentrar el apoyo en un único pedido.
5. Ofrecer a los funcionarios una bandeja por área para responder y registrar el avance hasta la resolución.
6. Generar estadísticas públicas anónimas y reportes exportables (Excel y PDF) para organismos.
7. Garantizar el anonimato entre vecinos y la protección de datos personales mediante reglas aplicadas en la base de datos.
8. Asegurar la calidad con pruebas automáticas, accesibilidad y buen rendimiento en celulares.

3. Contenido del trabajo práctico

3.1 Contexto, estudio de necesidades y reorientación

Análisis socioeconómico de Córdoba y encuesta

El equipo realizó previamente un análisis socioeconómico de la ciudad de Córdoba y una encuesta de necesidades a vecinos. Ambos se mantienen como base del proyecto, ya que justifican las categorías de problemáticas, el foco en el nivel de barrio y la necesidad de un canal ordenado de reclamos.

> Incorporar aquí los resultados del análisis socioeconómico y de la encuesta del equipo (preguntas, cantidad de respuestas y principales hallazgos). No se reproducen en esta versión para no alterar los datos originales.

Necesidades detectadas

- **Vecinos:** no tienen un canal único para reclamar ni saben si alguien atiende su pedido; reclaman en forma individual y sin seguimiento.
- **Barrios:** los mismos problemas se reportan muchas veces y de manera dispersa, sin un registro común que muestre su magnitud.
- **Organismos y ONG:** carecen de información ordenada por zona, urgencia y apoyo para decidir qué atender primero y rendir cuentas de lo hecho.

Reorientación del proyecto

Se mantiene	Se reemplaza
Análisis socioeconómico de Córdoba	Módulo de pagos y donaciones → **sistema de peticiones**
Arquitectura en capas (MVC)	Lógica de recaudación → **firmas y prioridad**
Base de datos relacional SQL	Seguimiento de fondos → **seguimiento de la respuesta oficial**
Autenticación y seguridad	Verificación de causas → **moderación y detección de similares**
Enfoque de evaluación de impacto	Informes de recaudación → **reportes para organismos**
Encuesta y estudio de necesidades	— (se agregan clasificación automática y score de urgencia)

3.2 Project Charter

Nombre del proyecto	ENTS — Everyone Needs To Smile
Descripción	Plataforma web de peticiones vecinales para Córdoba y Villa Allende, con clasificación automática, firmas, score de urgencia, detección de similares, seguimiento de la respuesta oficial y reportes para organismos.

Propósito / justificación	Dar a los vecinos un canal único y transparente para visibilizar problemas de su barrio y a los organismos información ordenada para priorizarlos, con la respuesta registrada a la vista de todos.
Objetivos medibles	• Plataforma publicada y accesible desde la web y como app instalable. • 100% de las peticiones nuevas con categoría, barrio y score asignados automáticamente. • Cobertura de los 111 barrios cargados (Córdoba y Villa Allende). • Pruebas automáticas aprobadas antes de cada publicación.
Alcance (resumen)	Registro e ingreso; peticiones con fotos y ubicación; firmas; procesamiento automático; mapa por barrios; comunidad y eventos; seguimiento y notificaciones; moderación; roles; estadísticas y reportes. Ver 3.3.
Entregables principales	Aplicación web publicada; base de datos con esquema documentado; repositorio de código; documentación del proyecto; presentación.
Interesados	Vecinos de Córdoba y Villa Allende; áreas del municipio y funcionarios; ONG y organizaciones sociales; docente de la cátedra; equipo de desarrollo.
Equipo	Jiménez, Ignacio · Andrada, Juan Pablo · Patrón, Máximo
Recursos	Humanos: 3 integrantes. Tecnológicos: GitHub (repositorio), Supabase (base de datos, autenticación y archivos), Netlify (publicación), OpenStreetMap (mapas), planes gratuitos. Costo monetario directo: sin costo con los planes gratuitos actuales.
Hitos	Reorientación aprobada · Diseño de datos · Primera versión funcional · Sistema de roles y anonimato · Procesamiento y reportes · Calidad (pruebas, rendimiento) · Entrega final.
Riesgos principales	Baja participación de vecinos al inicio · Contenido inapropiado o con datos personales · Límites de los planes gratuitos · Dependencia de servicios externos · Disponibilidad del equipo.
Supuestos	Los usuarios cuentan con acceso a internet y un navegador moderno; los organismos designan funcionarios para responder.
Restricciones	Plazo del ciclo lectivo; equipo de 3 personas; sin presupuesto; sin integración con sistemas internos del municipio.
Criterios de éxito	El ciclo completo (pedir → firmar → priorizar → responder → resolver) funciona de punta a punta en el sitio publicado, con las reglas de privacidad verificadas.
Aprobación	Docente: _____ Fecha: ___/___/_____

3.3 Declaración de alcance (Scope Statement)

Descripción del producto

Aplicación web adaptable a celular e instalable (PWA), con un panel para vecinos, una bandeja para funcionarios y paneles de moderación y análisis para administradores, respaldada por una base de datos relacional con reglas de seguridad por fila.

Incluido en el alcance

- Registro, ingreso con correo o Google y recuperación de contraseña.
- Peticiones con hasta 3 fotos y ubicación en mapa.
- Categoría y barrio automáticos; score de urgencia automático.
- Firmas, prioridad por firmas y meta de 50 firmas (elevación).
- Detección de peticiones similares.
- Moderación previa a la publicación y revisión automática de textos.
- Seguimiento en 5 etapas con respuesta oficial y foto.
- Comentarios anónimos con denuncias.
- Notificaciones dentro de la app y barrios seguidos.
- Mapa por barrios, comunidad vecinal y agenda de eventos.
- Estadísticas públicas, panel analítico y exportación a Excel y PDF.
- Roles: vecino, funcionario por área, administrador y dueño.

Fuera del alcance

- Pagos, donaciones o recaudación dentro de la plataforma (si existen, se realizan por fuera).
- Integración con sistemas internos del municipio.
- Envío de notificaciones por correo o SMS.
- Aplicación nativa de tienda (se cubre con la PWA).
- Polígonos oficiales de barrios (se usan centros de OpenStreetMap).

Criterios de aceptación

- Cada funcionalidad incluida operativa en el sitio publicado.
- Un vecino no puede ver identidades ajenas ni datos de otros.
- Pruebas automáticas aprobadas y compilación sin errores.

Supuestos y restricciones

Se trabaja con los planes gratuitos de los servicios elegidos, un equipo de tres integrantes y el calendario de la cátedra. La información geográfica de los barrios proviene de OpenStreetMap.

3.4 Estructura de desglose del trabajo (EDT / WBS)

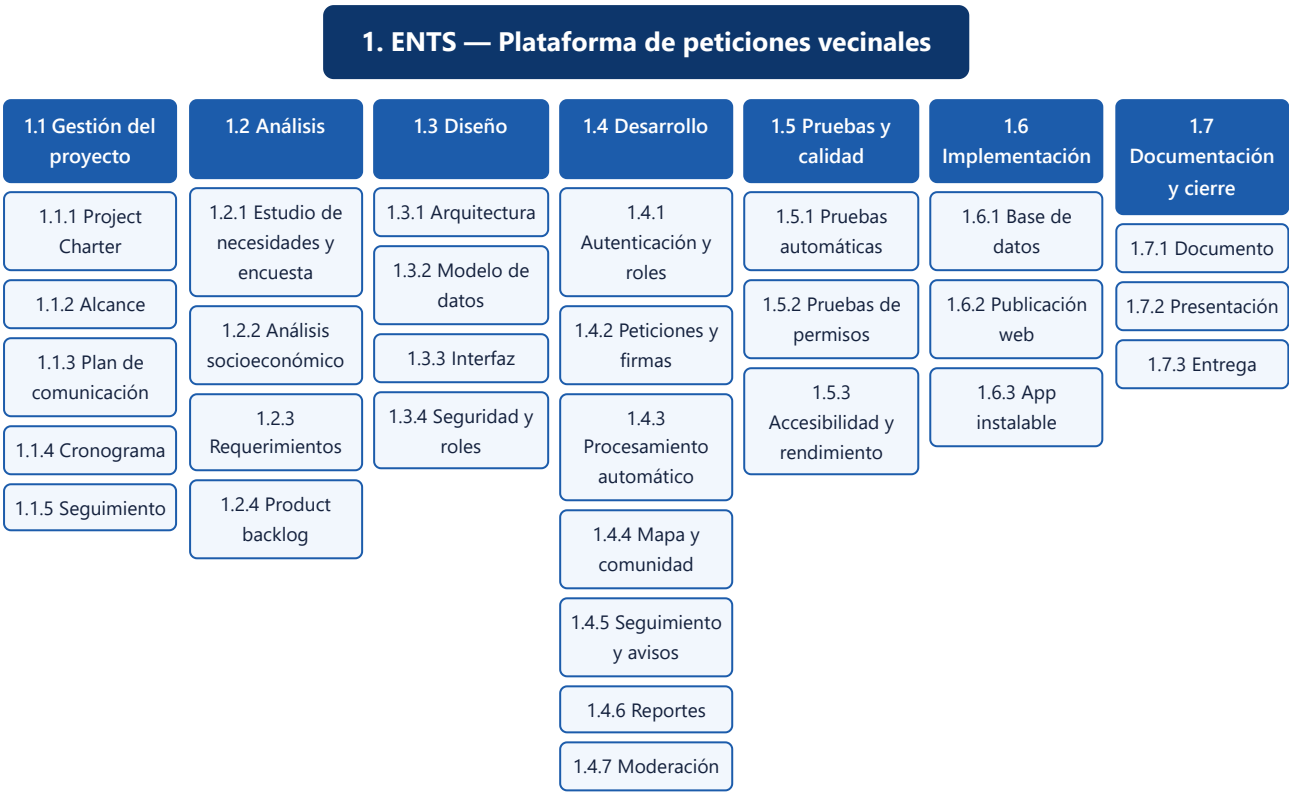


Figura 1. EDT del proyecto orientada a entregables (tres niveles).

3.5 Lista de actividades

Duraciones en días hábiles de trabajo del equipo. Las predecesoras indican qué actividad debe terminar antes de empezar la siguiente.

ID	Actividad	EDT	Duración	Predecesoras	Responsable
A1	Analizar la propuesta de reorientación y redefinir el proyecto	1.1.1	3	—	Equipo
A2	Redactar Project Charter y declaración de alcance	1.1.1–1.1.2	3	A1	Equipo
A3	Definir plan de comunicación y cronograma	1.1.3–1.1.4	2	A2	Equipo
A4	Revisar encuesta y análisis socioeconómico	1.2.1–1.2.2	3	A1	Equipo
A5	Relevar requerimientos y armar product backlog	1.2.3–1.2.4	4	A2, A4	Equipo
A6	Diseñar arquitectura y elegir tecnologías	1.3.1	3	A5, A3	Equipo
A7	Diseñar modelo de datos y reglas de seguridad	1.3.2, 1.3.4	4	A6	Equipo
A8	Diseñar interfaz y navegación	1.3.3	4	A5	Equipo

ID	Actividad	EDT	Duración	Predecesoras	Responsable
A9	Configurar repositorio, base de datos y publicación	1.6.1–1.6.2	2	A6	Equipo
A10	Desarrollar autenticación, roles y anonimato	1.4.1	5	A7, A9	Equipo
A11	Desarrollar peticiones, fotos, firmas y moderación	1.4.2, 1.4.7	6	A10, A8	Equipo
A12	Desarrollar clasificación, barrio, score y similares	1.4.3	5	A11	Equipo
A13	Desarrollar mapa por barrios, comunidad y eventos	1.4.4	5	A11	Equipo
A14	Desarrollar seguimiento, notificaciones y comentarios	1.4.5	5	A11	Equipo
A15	Desarrollar estadísticas, panel analítico y exportes	1.4.6	4	A12, A13	Equipo
A16	Pruebas automáticas y pruebas de permisos por rol	1.5.1–1.5.2	4	A12, A14, A15	Equipo
A17	Accesibilidad, modo oscuro, app instalable y rendimiento	1.5.3, 1.6.3	4	A15	Equipo
A18	Redactar documentación	1.7.1	5	A16, A17	Equipo
A19	Preparar presentación	1.7.2	2	A18	Equipo
A20	Entrega y defensa	1.7.3	1	A19	Equipo

> Completar la columna "Responsable" con el integrante a cargo de cada actividad según la distribución real del equipo.

3.6 Diccionario de actividades

EDT	Paquete	Descripción	Entregable	Criterio de aceptación
1.1	Gestión del proyecto	Planificación, coordinación y control del proyecto.	Charter, alcance, plan de comunicación, Gantt	Aprobados por el equipo y la cátedra
1.2.1	Estudio de necesidades	Encuesta a vecinos y análisis de problemáticas por barrio.	Informe de resultados	Necesidades priorizadas y vinculadas a categorías
1.2.3	Requerimientos	Relevamiento de requerimientos funcionales y no funcionales.	Lista RF/RNF	Cada RF trazable a una historia de usuario
1.2.4	Product backlog	Historias de usuario con criterios de aceptación y prioridad.	Backlog priorizado	Historias estimables y verificables
1.3.1	Arquitectura	Definición de capas, tecnologías y despliegue.	Diagrama de arquitectura y despliegue	Cubre todos los RF y RNF

EDT	Paquete	Descripción	Entregable	Criterio de aceptación
1.3.2	Modelo de datos	Tablas, relaciones, restricciones y funciones de la base.	DER y esquema SQL	Esquema reproducible desde `schema.sql`
1.3.4	Seguridad y roles	Reglas de acceso por fila y por columna, roles y anonimato.	Políticas RLS y funciones	Pruebas por rol sin accesos indebidos
1.4.1	Autenticación y roles	Registro, ingreso con Google, recuperación, solicitud de funcionario.	Módulo de cuentas	Flujos completos funcionando
1.4.2	Peticiones y firmas	Alta con fotos y ubicación, firma y retiro de firma, prioridad.	Módulo de peticiones	Un usuario firma una sola vez; contador correcto
1.4.3	Procesamiento automático	Clasificación, barrio, score de urgencia y similares.	Algoritmos y funciones	Casos de prueba resueltos correctamente
1.4.4	Mapa y comunidad	Mapa por barrios, muro vecinal y eventos.	Pantallas de mapa, comunidad y agenda	Filtros por barrio operativos
1.4.5	Seguimiento y avisos	Etapas, respuesta oficial, notificaciones, comentarios.	Línea de tiempo y campana	Cada cambio genera aviso a interesados
1.4.6	Reportes	Estadísticas públicas, panel analítico y exportes.	Paneles y archivos CSV/PDF	Datos coinciden con la base
1.4.7	Moderación	Aprobación, edición y revisión automática de contenido.	Panel de moderación	Nada se publica sin aprobación
1.5	Pruebas y calidad	Pruebas automáticas, de permisos, accesibilidad y rendimiento.	Suite de pruebas	100% de pruebas aprobadas
1.6	Implementación	Base de datos en la nube, publicación continua y PWA.	Sitio publicado	Cada versión publicada desde el repositorio
1.7	Documentación y cierre	Documento, presentación y entrega.	PDF y presentación	Cumple la estructura de la cátedra

3.7 Plan de comunicación

El plan asegura que el equipo, la cátedra y los usuarios piloto reciban la información correcta, en el momento adecuado y por el canal apropiado.

Qué se comunica	Emisor	Receptor	Canal	Frecuencia	Propósito
Avance y tareas del sprint	Equipo	Equipo	Reunión breve + grupo de mensajería	2 veces por semana	Coordinar y destrabar tareas
Cambios de código	Integrante	Equipo	GitHub (ramas y Pull Requests)	Por cambio	Revisión antes de publicar

Qué se comunica	Emisor	Receptor	Canal	Frecuencia	Propósito
Planificación de sprint	Equipo	Equipo	Tablero de tareas	Inicio de sprint	Priorizar el backlog
Revisión de sprint	Equipo	Docente	Clase / demostración	Fin de sprint	Validar avances
Cambios de alcance	Equipo	Docente	Reunión + documento	Cuando ocurren	Aprobar reorientaciones
Novedades de la plataforma	Plataforma	Vecinos	Notificaciones en la app	Automática	Informar estado de peticiones
Respuestas oficiales	Funcionarios	Vecinos	Seguimiento de la petición	Por actualización	Cerrar el ciclo de la petición
Reportes de problemáticas	Administración	Organismos y ONG	Exportes Excel / PDF	Periódica	Priorizar y rendir cuentas

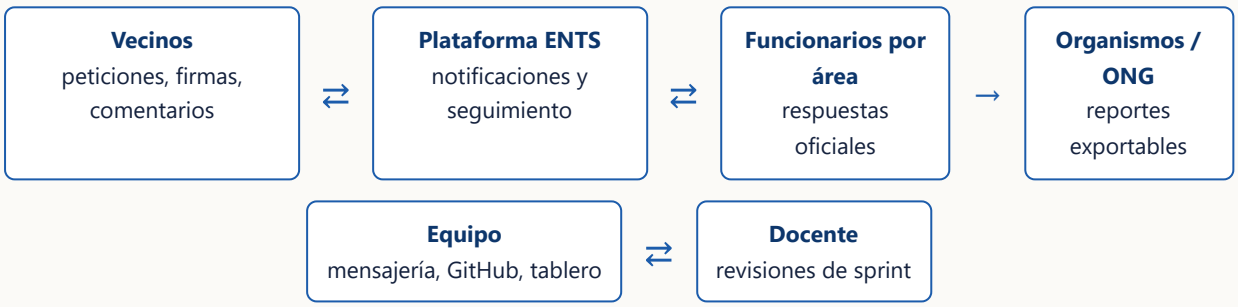
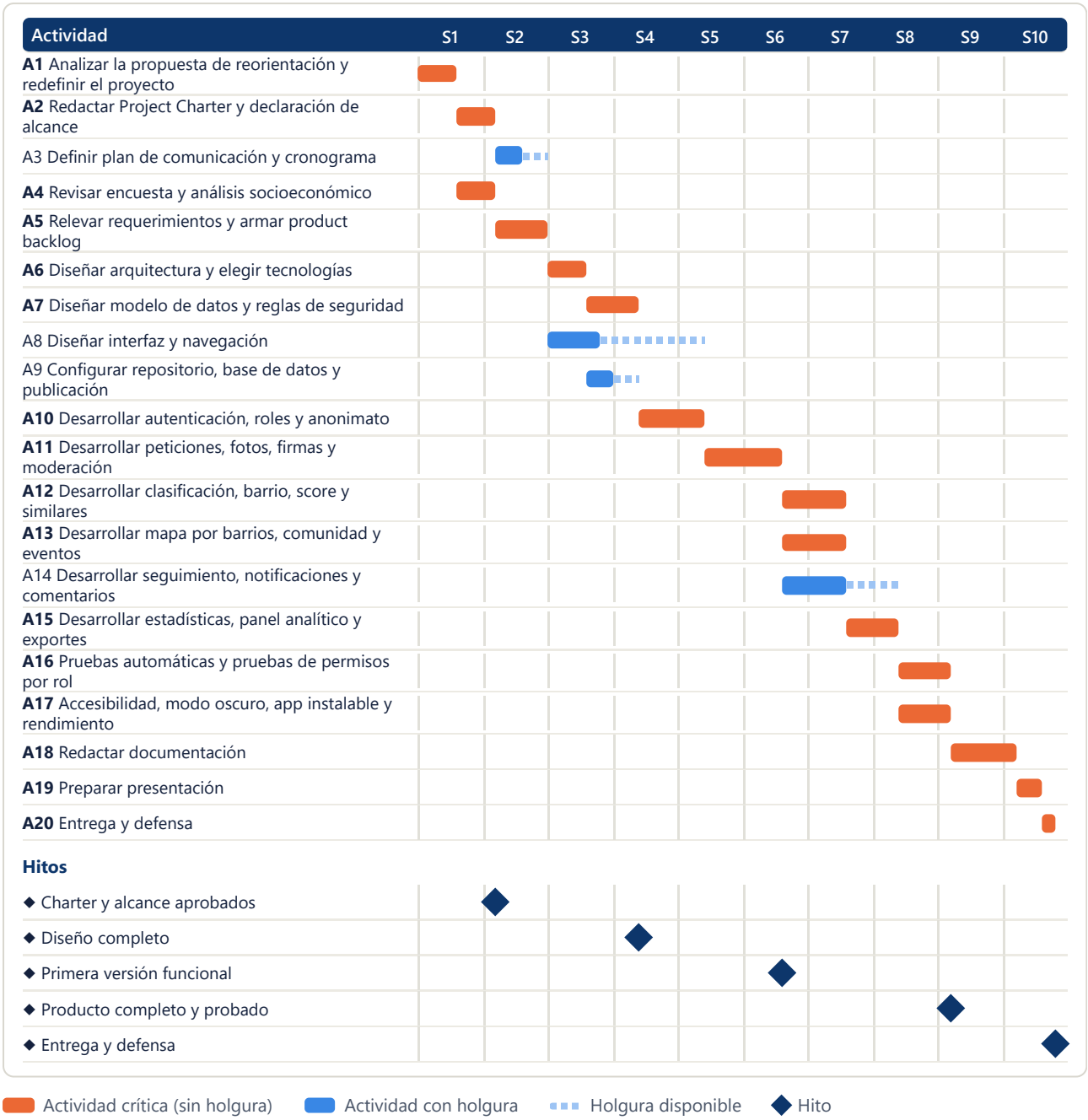


Figura 2. Flujo de la comunicación.

3.8 Diagrama de Gantt

Cronograma de 49 días hábiles (≈ 10 semanas) calculado a partir de la lista de actividades; en naranja, la ruta crítica. Las fechas reales de cada entrega de código figuran en el Anexo B.



Ruta crítica: A1 → A2 → A4 → A5 → A6 → A7 → A10 → A11 → A12 → A13 → A15 → A16 → A17 → A18 → A19 → A20.

3.9 Requerimientos y product backlog

Requerimientos funcionales

ID	Requerimiento
RF01	Registrar usuarios con correo y contraseña o con Google; recuperar la contraseña por correo.
RF02	Elegir el tipo de cuenta al registrarse (vecino o funcionario, este último sujeto a aprobación).
RF03	Crear peticiones con título, descripción, hasta 3 fotos y ubicación en el mapa.
RF04	Sugerir automáticamente la categoría según el texto y el barrio según la ubicación.

ID	Requerimiento
RF05	Calcular automáticamente el score de urgencia de cada petición.
RF06	Advertir peticiones similares antes de crear una nueva.
RF07	Moderar las peticiones antes de publicarlas (aprobar, rechazar, editar, eliminar).
RF08	Firmar y retirar la firma de una petición publicada; ordenar por prioridad según firmas.
RF09	Elevar automáticamente al área correspondiente las peticiones que alcanzan 50 firmas.
RF10	Registrar respuestas oficiales y la etapa de seguimiento (recibida → resuelta) con foto.
RF11	Notificar a autor y firmantes de cada novedad; permitir seguir barrios.
RF12	Comentar en forma anónima, denunciar comentarios y moderarlos.
RF13	Mostrar el mapa por barrios coloreado por peticiones, firmas o urgencia.
RF14	Ofrecer estadísticas públicas anónimas y un panel analítico para administradores.
RF15	Exportar reportes de peticiones y barrios a Excel y PDF.
RF16	Asignar áreas a funcionarios y mostrarles una bandeja de su área.
RF17	Permitir solo al dueño del proyecto cambiar el rol de las cuentas.
RF18	Publicar en el muro vecinal y consultar la agenda de eventos por barrio.

Requerimientos no funcionales

ID	Requerimiento
RNF01	Privacidad: anonimato entre vecinos; identidades visibles solo para funcionarios y administradores.
RNF02	Seguridad: reglas de acceso aplicadas en la base de datos, comunicación por HTTPS.
RNF03	Usabilidad: interfaz en español, adaptable a celular, modo claro y oscuro.
RNF04	Accesibilidad: navegación con teclado, foco visible, respeto de movimiento reducido, tablas alternativas a los gráficos.
RNF05	Rendimiento: carga por partes de cada página y animaciones reducidas en celulares.
RNF06	Disponibilidad: aplicación instalable (PWA) que abre aun con conexión inestable.
RNF07	Mantenibilidad: código con revisión automática (lint), pruebas automáticas y esquema SQL versionado.

Product backlog (historias de usuario)

#	Historia de usuario	Criterios de aceptación	Prioridad
HU1	Como vecino, quiero registrarme e ingresar (también con Google) para participar en la plataforma.	Formulario con nombre, apellido, correo y contraseña; mensajes de error en español; recuperación de contraseña.	Alta

#	Historia de usuario	Criterios de aceptación	Prioridad
HU2	Como vecino, quiero crear una petición con fotos y ubicación para visibilizar un problema de mi barrio.	Categoría y barrio sugeridos solos; hasta 3 fotos; queda pendiente de moderación.	Alta
HU3	Como vecino, quiero ver si ya existe un pedido parecido para firmarlo en lugar de repetirlo.	Se muestran hasta 3 similares mientras escribo, con barrio, firmas y distancia.	Alta
HU4	Como vecino, quiero firmar una petición para darle más prioridad.	Una firma por persona; puedo retirarla; el orden y el color de prioridad se actualizan.	Alta
HU5	Como administrador, quiero moderar las peticiones antes de publicarlas.	Aprobar, rechazar, editar o eliminar; alertas automáticas visibles.	Alta
HU6	Como funcionario, quiero ver las peticiones de mi área ordenadas por prioridad y responderlas.	Bandeja filtrada por área; respuesta con etapa, mensaje y foto.	Alta
HU7	Como vecino, quiero seguir el avance de una petición hasta que se resuelva.	Línea de tiempo con 5 etapas y respuestas oficiales; aviso en cada cambio.	Alta
HU8	Como vecino, quiero ver en un mapa los problemas por barrio.	Mapa con 111 barrios coloreados; filtro y botón para seguir un barrio.	Media
HU9	Como organismo, quiero reportes de qué se pide, dónde y con qué urgencia.	Estadísticas por categoría, barrio y semana; exportes a Excel y PDF.	Media
HU10	Como vecino, quiero comentar en forma anónima para aportar datos.	Se bloquean datos personales; denuncia y ocultamiento con 3 denuncias.	Media
HU11	Como vecino, quiero gestionar mi perfil y ver mis logros.	Edición de nombre y foto; insignias; barrios seguidos.	Baja
HU12	Como dueño del proyecto, quiero asignar el rol de cada cuenta.	Solo el dueño ve y puede usar el selector de rol.	Baja

3.10 Arquitectura del sistema

Se mantiene la separación en capas del diseño original (modelo-vista-controlador), pero se simplifica la infraestructura: en lugar de un backend propio en Node.js/C# con MySQL, Docker y Heroku, se utiliza un **backend como servicio** (Supabase, sobre PostgreSQL) que provee base de datos, autenticación, almacenamiento de archivos y una API generada a partir del esquema. Esto reduce la infraestructura a mantener y concentra la lógica de negocio crítica en la base de datos, donde no puede ser alterada desde el navegador.

Capa	Tecnología	Responsabilidad
Vista	React 19, React Router, Bootstrap 5 con tema propio, Leaflet	Páginas y componentes; interacción; validaciones inmediatas (categoría, moderación, similares).
Controlador	Servicios de la app (`src/services`) y funciones RPC de la base	Orquestación de casos de uso: crear, firmar, responder, moderar, reportar.
Modelo	PostgreSQL en Supabase, con RLS y disparadores	Datos, reglas de acceso, cálculo del score, notificaciones, prioridad y elevación.

Paquetes del código

Paquete	Contenido
`src/pages`	Una pantalla por archivo: Inicio, Mapa, Comunidad, Eventos, Petición, Nueva petición, Bandeja, Estadísticas, Moderación, Usuarios, Perfil, Acceso.
`src/components`	Piezas reutilizables: tarjeta de petición, seguimiento, comentarios, gráficos, selectores de fotos y ubicación, menús.
`src/services`	Acceso a datos: peticiones, participación (seguimiento, comentarios, roles) y autenticación.
`src/utils`	Lógica pura y probada: prioridad, clasificador, barrios, moderación, exportación, zonas del mapa.
`supabase/schema.sql`	Esquema completo: tablas, políticas, funciones y disparadores.

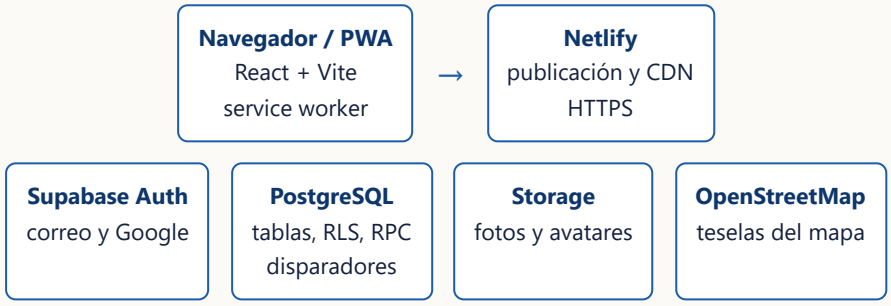


Figura 3. Diagrama de despliegue: el cliente se descarga desde Netlify y se comunica por HTTPS con los servicios de Supabase y con OpenStreetMap. GitHub dispara la publicación automática en cada versión.

3.11 Modelo de datos

Base relacional en PostgreSQL con 13 tablas. Todas tienen seguridad por fila activada.



Figura 4. Diagrama entidad-relación (simplificado). Relaciones principales: una petición pertenece a un usuario, una categoría y un barrio; tiene muchas firmas, actualizaciones y comentarios; cada categoría pertenece a un área.

Diagrama de estados de una petición

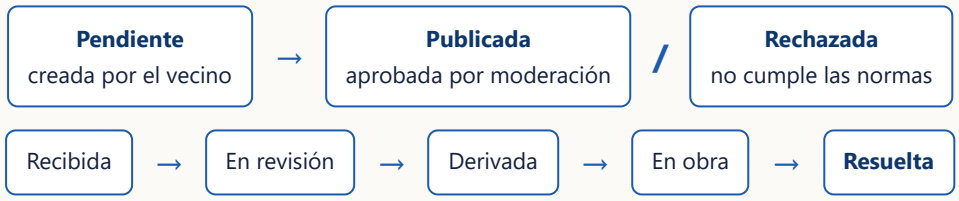


Figura 5. Estados de moderación (arriba) y etapas de seguimiento de una petición publicada (abajo). Al alcanzar 50 firmas se registra además la elevación al área.

3.12 Procesamiento automático de datos

Clasificación por categoría

Cada una de las 16 categorías tiene raíces de palabras clave (por ejemplo, Salud: hospital, dispensario, pediatr…). El texto se normaliza (minúsculas, sin acentos) y se cuentan coincidencias al inicio de palabra; las del título valen el doble. Gana la categoría con más puntaje; si hay empate total, no se sugiere y elige la persona.

Detección del barrio

Con la ubicación marcada (o la del dispositivo) se elige el barrio cuyo centro está más cerca, hasta 3 km, entre 111 barrios de Córdoba y Villa Allende cuyas coordenadas se obtuvieron de OpenStreetMap.

Score de urgencia (0 a 100)

Componente	Puntos
Gravedad de la categoría (p. ej. Salud 35, Cultura 12)	hasta 35

Componente	Puntos
Palabras de riesgo en el texto (peligro, accidente, inundación, sin agua, chicos…)	8 c/u, hasta 30
Apoyo: 20 × ln(1 + firmas) / ln(51)	hasta 20
Velocidad: 2 por firma de los últimos 7 días	hasta 10
Evidencia fotográfica	5

Se calcula en la base de datos al crear, editar y con cada firma. Niveles: urgente (70 o más), media (40 a 69) y baja (menos de 40). Ejemplo: "Cables caídos, peligro para los chicos" (Servicios públicos) obtiene 53; con 10 firmas y una foto llega a 70.

Prioridad por firmas

Nueva (0–4), baja (5–9), media (10–19), alta (20–49) y máxima (50 o más). El inicio ordena por firmas, luego por urgencia y por fecha. A las 50 firmas la petición se eleva y se notifica a los funcionarios del área y a los administradores.

Detección de peticiones similares

Se extraen las palabras clave (sin conectores ni términos genéricos como "calle" o "barrio") y se comparan por similitud de trigramas (extensión `pg_trgm`). Se sugieren hasta 3 peticiones con parecido ≥ 0,32, o ≥ 0,22 si están a menos de 800 metros.

Moderación automática

Se detectan correos, teléfonos, números de documento y lenguaje ofensivo. En comentarios, los datos personales se bloquean y el lenguaje ofensivo queda oculto hasta revisión; en peticiones, se muestran alertas al moderador.

3.13 Autenticación, roles y seguridad

Rol	Permisos
Vecino	Crear, firmar y comentar peticiones; seguir barrios; ver estadísticas públicas. Anónimo para los demás vecinos.
Funcionario	Ver autor y firmantes; responder y cambiar la etapa de las peticiones de su área. Lo aprueba un administrador.
Administrador	Moderar peticiones y comentarios, asignar áreas, ver el directorio de usuarios y el panel analítico.
Dueño	Único que puede cambiar el rol de las cuentas.

- **Autenticación:** correo y contraseña o Google (OAuth); recuperación de contraseña por correo; sesiones gestionadas por Supabase Auth.
- **Seguridad por fila (RLS):** cada tabla define quién puede leer, crear, modificar o borrar cada fila.
- **Anonimato por columna:** la columna del autor de una petición no es legible para los vecinos; las identidades solo se obtienen mediante funciones que verifican el rol.

- **Funciones seguras:** las operaciones sensibles (responder, moderar, cambiar rol, estadísticas de administración) verifican el rol dentro de la base.
- **Validaciones del servidor:** el score, la etapa inicial y las alertas los fija la base, aunque el cliente intente enviar otros valores.
- **Transporte:** toda la comunicación usa HTTPS.

Las reglas se verificaron simulando usuarios de cada rol dentro de transacciones que luego se deshacen: un vecino no puede responder como funcionario, ni publicar un teléfono, ni cambiar la etapa, ni ver la moderación, ni cambiar roles.

3.14 Reportes y evaluación de impacto

La evaluación de impacto se apoya en los datos que la plataforma registra en cada paso del ciclo:

Indicador	Cómo se mide
Problemáticas activas	Peticiones publicadas por categoría, barrio y semana.
Apoyo comunitario	Firmas totales, por día y velocidad de crecimiento.
Urgencia	Distribución alta / media / baja y urgencia promedio por barrio.
Respuesta institucional	Peticiones por etapa de seguimiento y tiempo promedio de revisión.
Resolución	Cantidad y proporción de peticiones resueltas.
Participación	Cuentas activas en 30 días, publicaciones y comentarios.

Las estadísticas públicas muestran solo agregados anónimos. El panel analítico de administradores agrega datos de cuentas y moderación, y permite exportar las peticiones (con área, barrio, etapa, prioridad, firmas y urgencia) y el resumen por barrio a Excel, y la vista completa a PDF, para presentar ante organismos y ONG.

3.15 Pruebas y calidad

- **Pruebas automáticas:** 43 pruebas con Vitest sobre prioridad, orden del inicio, clasificación, detección de barrio, moderación de textos, áreas del mapa y exportación. Las pruebas detectaron y permitieron corregir un error real en la detección de lenguaje ofensivo.
- **Pruebas de permisos:** escenarios por rol ejecutados en la base de datos.
- **Revisión de código:** oxlint sin errores y compilación de producción antes de cada publicación.
- **Accesibilidad:** enlace para saltar al contenido, foco visible, etiquetas asociadas a los campos, contraste en modo claro y oscuro, versión en tabla de cada gráfico y respeto de la preferencia de movimiento reducido.
- **Rendimiento:** cada página se descarga recién al visitarla; en celulares se desactivan animaciones continuas y desenfoques; las listas largas no se dibujan fuera de pantalla.

3.16 Ambiente de implementación y despliegue

Herramienta	Uso
React 19 + Vite 8	Desarrollo y compilación del cliente web.
Bootstrap 5 + tema propio	Estilos, modo claro y oscuro.
Leaflet + OpenStreetMap	Mapa vecinal y selector de ubicación.
Supabase (PostgreSQL)	Base de datos, autenticación, almacenamiento y API.
Netlify	Publicación continua desde GitHub con HTTPS.
Git + GitHub	Control de versiones; ramas y Pull Requests.
Vitest + oxlint	Pruebas automáticas y revisión de código.
Node.js 22	Entorno de desarrollo local.

Ambientes: desarrollo local (`npm run dev`) conectado a la base en la nube; producción en Netlify, que publica automáticamente cada versión de la rama principal del repositorio. El esquema completo de la base se versiona en `supabase/schema.sql` para poder recrearla.

4. Bibliografía

Obra	Autor	Edición	Editorial
Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK®), 7.ª edición	Project Management Institute	2021	Project Management Institute
Ingeniería de Software, 9.ª edición	Ian Sommerville	2011	Pearson Educación
Ingeniería del software: un enfoque práctico, 7.ª edición	Roger S. Pressman	2010	McGraw-Hill
La Guía de Scrum	Ken Schwaber y Jeff Sutherland	2020	Scrum.org / Scrum Inc.
Clean Code: A Handbook of Agile Software Craftsmanship	Robert C. Martin	2008	Prentice Hall
Web Content Accessibility Guidelines (WCAG) 2.2	W3C	2023	World Wide Web Consortium

5. Fuentes de información

- Docentes de la cátedra Práctica Profesionalizante del Instituto Cervantes.
- Documentación previa del equipo: presentación "Everyone Needs To Smile", entregables de gestión y documento de Práctica Profesionalizante (versión de donaciones).
- Encuesta de necesidades a vecinos y análisis socioeconómico de Córdoba realizados por el equipo.
- OpenStreetMap y su servicio de búsqueda Nominatim: ubicación de los 111 barrios de Córdoba y Villa Allende.
- Documentación oficial de React (react.dev), Vite (vite.dev), Supabase (supabase.com/docs), PostgreSQL (postgresql.org/docs), Leaflet (leafletjs.com), Netlify (docs.netlify.com) y MDN Web Docs (aplicaciones web progresivas).

6. Anexo

A. Glosario

Petición	Pedido de un vecino sobre un problema de su barrio.
Firma	Apoyo de un vecino a una petición; cada persona firma una sola vez.
Score de urgencia	Puntaje de 0 a 100 calculado automáticamente según gravedad, riesgo, apoyo y evidencia.
Elevación	Aviso automático al área cuando una petición alcanza 50 firmas.
RLS	Row Level Security: reglas de la base que deciden qué filas puede ver o modificar cada usuario.
RPC	Función de la base de datos invocada desde la aplicación.
PWA	Aplicación web progresiva: sitio que se puede instalar como app.

B. Hitos reales del repositorio

Fecha	Entrega
04/10/2026	Configuración inicial del proyecto; autenticación, peticiones, firmas y moderación.
11/10/2026	Panel vecinal conectado a Supabase; roles y anonimato; ingreso con Google y recuperación de contraseña; política de privacidad.
12/10/2026	Estadísticas y mapa por barrios; 111 barrios; prioridad por firmas, seguimiento, comentarios y notificaciones; bandeja por área, exportes, PWA, modo oscuro y pruebas; cambio de roles; rendimiento en celulares; score de urgencia automático.

C. Fórmula del score de urgencia (extracto del esquema)

```
gravedad  := peso_urgencia_de_la_categoria * 35 / 30;          -- hasta 35
riesgo    := least(8 * palabras_de_riesgo_encontradas, 30);    -- hasta 30
apoyo     := least(20, 20 * ln(1 + firmas) / ln(51));          -- hasta 20
velocidad := least(10, 2 * firmas_de_los_ultimos_7_dias);      -- hasta 10
evidencia := case when tiene_fotos then 5 else 0 end;          -- 5
score     := least(100, round(gravedad + riesgo + apoyo + velocidad + evidencia));
```

D. Puesta en marcha local

```
git clone https://github.com/andradaj09/ENTS.git
cd ENTS
npm install
# crear .env con VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY
npm run dev        # http://localhost:5173
npm test           # pruebas automáticas
npm run build      # versión de producción
```

E. Enlaces

- Aplicación publicada: https://entscordoba.netlify.app
- Repositorio (privado): https://github.com/andradaj09/ENTS